

DEVOPS E ARQUITETURA CLOUD

FASE 2 | AULA 08 -

ESCALABILIDADE AVANÇADA DE APLICAÇÕES COM KEDA

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	12
O QUE VOCÊ VIU NESTA AULA?	12
REFERÊNCIAS.....	18

EMANIP

O QUE VEM POR AÍ?

O KEDA (Kubernetes Event-driven Autoscaler) é uma solução que complementa o HPA tradicional do Kubernetes, permitindo escalonamento com base em eventos externos, como filas ou métricas customizadas. Diferente do HPA, ele permite escalar aplicações até zero réplicas quando não há carga, economizando recursos. Quando um evento é detectado, o KEDA ativa os pods automaticamente e trabalha em conjunto com o HPA para escalar conforme necessário. É ideal para workloads event-driven e cenários serverless, oferecendo escalabilidade sob demanda de forma mais eficiente que o modelo baseado apenas em CPU/memória.

HANDS ON

Para ilustrar o uso do KEDA na prática, vamos explorar a criação de um **ScaledObject**, o recurso personalizado (CRD) central do KEDA. Um **ScaledObject** define qual aplicação será escalada (via referência a um Deployment alvo) e **qual trigger externo** dirigir o escalonamento.

Suponha que temos um deployment chamado "meu-deployment" rodando um worker que processa mensagens de uma fila RabbitMQ chamada "minha-fila". Veja o passo a passo completo nas videoaulas!

SAIBA MAIS

Em um ambiente orientado a eventos, queremos que os pods dos Workers do Kubernetes escalem automaticamente conforme o tamanho da fila – por exemplo, 1 pod para cada 10 mensagens pendentes – podendo chegar a zero pods quando a fila estiver vazia.

Com o cliente Kubernetes em **Rust** (crate kube + bindings do KEDA), podemos definir esse ScaledObject programaticamente conforme abaixo:

```
usekube::{Client, Api, api::PostParams};
usekcr_keda_sh::v1alpha1::ScaledObject;
usekcr_keda_sh::v1alpha1::scaledobjects::{ScaledObjectSpec, ScaledObjectTriggers,
ScaledObjectScaleTargetRef};

// Conecta ao cluster Kubernetes (assume KUBECONFIG definido)
letclient=Client::try_default().await?;
letscaledobjects=Api<ScaledObject>=Api::namespaced(client, "default");

// Define o trigger do RabbitMQ com parâmetros
lettrigger=ScaledObjectTriggers {
    type_:"rabbitmq".into(), // tipo do scaler: RabbitMQ
    metadata:vec![
        ("queueName".into(), "minha-fila".into()), // nome da fila RabbitMQ
        ("protocol".into(), "amqp".into()), // protocolo de conexão (AMQP)
        ("mode".into(), "QueueLength".into()), // escalar com base no tamanho
da fila
        ("value".into(), "10".into()) // 10 mensagens por pod
    ].into_iter().collect(),
    authentication_ref:None, // credenciais (poderiam ser referenciadas via
TriggerAuthentication)
    ..Default::default()
};

// Especifica a referência do Deployment alvo e parâmetros de escalonamento
letspec=ScaledObjectSpec {
    scale_target_ref:ScaledObjectScaleTargetRef {
        name:"meu-deployment".into(),
        ..Default::default()
    },
    triggers:vec![ trigger ],
    min_replica_count:Some(0), // pode escalar até zero pods
    max_replica_count:Some(5), // máximo de 5 pods
    cooldown_period:Some(60), // espera 60s de inatividade antes de escalar
para baixo
    polling_interval:Some(15), // checa a fila a cada 15s
    ..Default::default()
};

// Cria o ScaledObject no cluster Kubernetes
letsobj=ScaledObject::new("meu-scaledobject", spec);
scaledobjects.create(&PostParams::default(), &sobj).await?;
```

Código-fonte 1 – Linguagem Rust
Fonte: elaborado pelo autor (2025)

No código acima, instanciamos um `ScaledObject` chamado **meu-scaledobject** com um gatilho do tipo `RabbitMQ`. Esse gatilho informa ao KEDA que ele deve consultar, a cada 15 segundos, o tamanho da fila "minha-fila" no broker `RabbitMQ` (via protocolo `AMQP`). Vamos explorar este conteúdo durante a aula.

A propriedade `value: "10"` significa que desejamos um pod para cada 10 mensagens – em outras palavras, a métrica alvo para o HPA será 10 mensagens por pod. Definimos `min_replica_count = 0` permitindo escala até zero, e `max_replica_count = 5` limitando o escalonamento a no máximo 5 réplicas simultâneas. O `cooldown_period = 60` segundos estabelece que, após o último evento de ativação, o KEDA aguardará 1 minuto antes de escalar *de volta para zero*, evitando desligar pods prematuramente em caso de rajadas intermitentes. Vale notar que o HPA tradicional lida com escala **de 1 a N**; assim, durante atividade, o Deployment oscilará entre 1 e 5 pods conforme necessário, mas somente o KEDA tem autoridade para efetivamente desativar todos (ir para 0) após o período de `cooldown`.

Em termos de funcionamento, assim que aplicamos esse `ScaledObject`, o KEDA cria em background um HPA associado. Quando a fila `RabbitMQ` está vazia, o KEDA mantém o Deployment desativado (0 pods). Ao chegar pelo menos uma mensagem, o KEDA **ativa** imediatamente o Deployment para 1 pod (conceito de *activation threshold*, por default 0 mensagens já ativa).

Esse primeiro pod se inicia e começa a consumir mensagens da fila. Se o ritmo de chegada de mensagens aumentar e o comprimento da fila exceder 10, 20, 30, ... mensagens, o KEDA alimenta o HPA com a métrica customizada de **queue length**. O HPA então calcula o número desejado de pods para manter ~10 msgs/pod – por exemplo, 40 mensagens na fila implicariam 4 pods desejados. O Kubernetes inicia pods adicionais até atingir esse estado, escalando horizontalmente em questão de segundos ou minutos (conforme a latência de polling e tempo de scheduling dos pods).

O **fluxo inverso** ocorre ao esvaziar a fila: conforme o backlog de mensagens diminui, o HPA reduz gradualmente o número de réplicas (pods) em execução. Quando a fila zera completamente e permanece assim pelo período de `cooldown` configurado (60s), o KEDA **escala de volta para zero** removendo o último pod. Esse mecanismo garante que nenhum recurso fique consumido quando não há trabalho – um ganho claro em eficiência.

Podemos **observar os eventos de escalonamento** gerados para validar esse comportamento. O próprio HPA emite eventos no Kubernetes a cada ajuste, indicando, por exemplo, "Scaled up to 4 réplicas; reason: queueLength above target" (motivo: a métrica de comprimento de fila excedeu o alvo).

Esses eventos são visíveis via `kubectl describe hpa` ou programaticamente via API. Além disso, o objeto `ScaledObject` mantém em seu **status** campos informativos, como o `lastActiveTime` (última vez em que um trigger esteve ativo) e o nome do HPA gerado. É possível consultar via Kubernetes API esses dados de status para consolidar métricas de escalonamento – por exemplo, um controlador customizado poderia registrar timestamps de ativações ou contabilizar quantas vezes o scaler foi acionado em determinado período.

Toda a infraestrutura de métricas permanece **integrada**: o KEDA complementa o pipeline de métricas do Kubernetes, em vez de substituí-lo, possibilitando monitoramento unificado. Com poucas linhas de código (ou YAML), conseguimos configurar um autoscaling sob medida, acoplado a fontes externas e cobrindo desde scale-out agressivo até scale-in total, tudo orquestrado automaticamente pelo Kubernetes.

Teoria de Filas

Para aprofundar os conceitos, podemos modelar matematicamente o escalonamento dirigido por eventos como um problema de **teoria de filas**. Imagine que eventos chegam a uma taxa média λ (por exemplo, mensagens/segundo entrando na fila) e cada pod consegue processar eventos a uma taxa média μ (serviço por segundo). Com c pods em operação, a capacidade de processamento total é aproximadamente $c \times \mu$.

O comportamento do sistema de filas pode ser analisado como um modelo $M|M|c$ clássico (chegadas Poisson, tempos de serviço exponenciais, c servidores paralelos). Uma condição fundamental de **estabilidade** é que a taxa de chegada não exceda a capacidade: $\lambda < \frac{c}{\mu}$. Em termos do modelo, definimos a **utilização** $\rho = \frac{\lambda}{\mu}$ para um único servidor; para múltiplos servidores, requeremos $\frac{\rho}{c} < 1$ para que o sistema alcance estado estacionário.

Se λ ultrapassar $c \times \mu$, nem mesmo escalonamento horizontal conseguirá prevenir o acúmulo indefinido de backlog – a fila crescerá sem bound enquanto a taxa efetiva de saída for menor que a de entrada, caracterizando instabilidade. Portanto, um autoscaler bem-sucedido precisa, idealmente, ajustar c (número de pods) de modo que $c \times \mu$ acompanhe λ no tempo, mantendo $\rho \approx 1$ ou abaixo disso.

Esse acompanhamento não é instantâneo; há inerentemente um **atraso de resposta** entre detectar o aumento de λ e disponibilizar mais pods (pelo tempo de inicialização dos contêineres e de decisão do controlador). Assim, durante picos repentinos, algum nível de **backlog** é inevitável até que novos pods entrem em atividade.

A relação entre backlog e performance pode ser quantificada pela **Lei de Little**, um resultado fundamental da teoria das filas. **Little's Law** estabelece que, em estado estável, o número médio de itens na fila L é igual à taxa média de chegada λ multiplicada pelo tempo médio de permanência (espera + serviço) W no sistema:

$$L = \lambda \cdot W$$

Em outras palavras, se clientes (ou mensagens) chegam a 20 por segundo e cada um permanece em média 0,5 segundos sendo atendido, então em média $L = 20 \times 0.5 = 10$ itens estarão no sistema (fila + sendo processados) simultaneamente.

No contexto do KEDA, podemos interpretar L como o tamanho médio da fila (backlog), W como o tempo médio de processamento de uma mensagem (incluindo eventual espera na fila) e λ como a vazão de eventos. Essa lei fornece um *balanço* entre throughput e latência: se o backlog cresce, significa que ou as chegadas estão muito rápidas ou o processamento/pods insuficientes, resultando em maior W (maior tempo de resposta).

Rearranjando, $W = \frac{L}{\lambda}$ – ou seja, o **tempo médio de resposta** é proporcional ao backlog dividido pela taxa de serviço. Isso formaliza a ideia intuitiva de que filas maiores implicam maior latência para quem está esperando. Um autoscaler ideal busca controlar L (mantendo-o próximo de zero ou de um patamar aceitável) para limitar W a um valor alinhado com o **SLO** (objetivo de nível de serviço, como tempo máximo de resposta). Por exemplo, se desejamos tempo médio de processamento de 5 segundos e chegam 2 msgs/s, o Little nos diz que devemos manter $L \approx \lambda W =$

$2 \times 5 = 10$ mensagens na fila em média – quaisquer excedentes além disso indicariam violação do SLO, sinalizando necessidade de escalonamento.

Na prática, sistemas de autoscaling enfrentam desafios de **controle** que podem introduzir *jitter* e *oscilações* no número de pods. O jitter refere-se a pequenas flutuações frequentes (p.ex. subir de 5 para 6 pods e logo voltar para 5 repetidamente), enquanto a oscilação pode ser algo mais pronunciado como um comportamento de vai-e-volta constante (p.ex. alternar entre 1 e 10 pods de forma cíclica).

Esses fenômenos geralmente decorrem de *atrasos de realimentação* e de *thresholds estáticos*. O HPA básico calcula o número desejado de pods a cada 15 segundos usando médias momentâneas; se a carga está variando rapidamente dentro desse intervalo ou próximo do limiar, o resultado pode ser uma sequência de decisões contraditórias a cada iteração. Sem políticas de estabilização, o HPA **pode oscilar**, escalando para cima e para baixo com demasiada frequência em resposta a ruídos ou picos transitórios.

Para mitigar isso, tanto o HPA quanto o KEDA incorporam mecanismos de **histerese e estabilização temporal**. O Kubernetes por padrão aplica uma tolerância de ~10% nas métricas para não reagir a variações pequenas: por exemplo, só escala se a métrica exceder 110% do alvo ou cair abaixo de 90%, evitando “bateção” contínua de 1 pod para 2 pods por variações mínimas.

Além disso, desde versões recentes é possível configurar *políticas de comportamento* do HPA, impondo janelas de estabilidade (por exemplo, não reduzir pods mais que uma vez a cada X segundos, ou olhar a média dos últimos Y segundos para decisões de scale down). O KEDA complementa essas proteções adicionando o já mencionado `cooldownPeriod` no `scale-to-zero` e permitindo especificar **estratégias avançadas** via `ScaledObject.spec.advanced`, onde pode-se definir explicitamente comportamentos de scale-up/scale-down análogos a um controlador PID (Proporcional-Integral-Derivativo).

Em termos formais, estamos lidando com um sistema de controle de realimentação discreto, onde métricas amostradas periodicamente servem de entrada para decisões de correção (número de pods). Existem limites teóricos impostos pela frequência de amostragem e latência: se eventos flutuam numa escala de tempo mais rápida do que o intervalo de *polling* do autoscaler, este não conseguirá reagir a cada

oscilação individual – em vez disso, verá uma média ou atraso, possivelmente gerando **respostas defasadas**.

Esse atraso pode causar *overshooting* (escalonar acima do necessário quando finalmente reage a um acúmulo de eventos) ou *undershooting* (escalar para baixo tarde demais, matando pods só quando já não havia carga há um tempo, ou ainda removendo pods antes do fim de todos os trabalhos em processamento). Estudos de estabilidade de sistemas de filas mostram que para manter uma fila estável sem crescer, é preciso reagir suficientemente rápido para acomodar λ dentro de $c\mu$ na média; qualquer latência de reação adiciona momento na fila, que pode se converter em oscilações se o controlador for muito agressivo tentando zerar o erro.

Por isso, boas práticas incluem usar **filtros e amortecimento** nas métricas (por exemplo, médias móveis ou percentis) e limites nas rampas de escalonamento (ex.: aumentar no máximo 2x por minuto), que de fato já aparecem nas policies do HPA v2. Enxergar o autoscaler sob a ótica de teoria de controle e filas nos ajuda a compreender que um equilíbrio delicado precisa ser alcançado: reagir rápido o suficiente para manter o sistema estável e dentro dos SLOs, mas não tão rápido a ponto de amplificar ruído e introduzir instabilidade.

Outra consideração formal diz respeito ao escalonamento assíncrono e tolerância a *bursts* (rajadas súbitas de eventos). Sistemas robustos frequentemente adotam abordagens de **queue buffering** para lidar com bursts: ao invés de tentar escalar instantaneamente para cada pico (o que poderia ser inviável se picos duram segundos, menos que o tempo de iniciar novos pods), aceita-se que a fila aumente temporariamente – consumindo o burst com a capacidade atual – enquanto se inicia gradualmente pods extras.

Essa estratégia se beneficia do fato de que filas atuam como **amortecedores naturais**. Do ponto de vista formal, um burst de Δ eventos pode ser acomodado desde que haja capacidade ociosa na fila (digamos, backlog backlog até certo ponto tolerável) e que W resultante ainda esteja dentro do limite aceitável.

Em escalonamento assíncrono, muitas vezes definimos **gatilhos de alta e baixa com histerese**: por exemplo, escalar *up* somente quando a fila excede 100 mensagens, mas só escalar *down* quando cair abaixo de 20. Essa diferença impede ping-pong em torno de um mesmo valor.

Podemos modelar isso como dois limiares L_{high} e L_{low} para o backlog, introduzindo uma banda morta (deadband) onde não há mudanças. Além disso, a configuração de um cooldown prolongado (como os 60s do exemplo) é uma implementação simples dessa tolerância a bursts: garante-se que o sistema não reduza imediatamente após um pico, mantendo pods “quentes” caso um novo burst suceda logo em seguida.

Há também estratégias proativas baseadas em previsão (por exemplo, usar modelos de previsão ou padrões de horário para antecipar escalonamentos – o KEDA suporta triggers de cron para horários de pico conhecidos). Em todos os casos, o objetivo é assegurar **estabilidade** e **resiliência a picos**: matematicamente, evitar oscilações significa garantir que o sistema de controle do autoscaler tenha um **ganho** adequado (não supersensível) e um **atraso** suficientemente pequeno frente às dinâmicas de chegada, de modo que o ponto de equilíbrio (número de pods necessário) seja atingido sem grandes desvios.

Muitos dos resultados clássicos para filas M/M/1 e M/M/c, como tempo médio de espera W_q ou tamanho médio da fila L_q em regime estacionário, assumem um número fixo de servidores c . Com autoscaling, c torna-se variável no tempo – porém, enquanto a adaptação seja mais lenta que as flutuações instantâneas, podemos considerar períodos quase-estáticos onde c é constante e aplicar as fórmulas para estimar o backlog e o tempo de resposta naquele patamar.

Ferramentas como a Lei de Little continuam válidas localmente a cada ajuste de c . Assim, combinando teoria de filas com controle, conseguimos projetar melhor os parâmetros do KEDA (polling interval, cooldown, thresholds) para garantir que operemos dentro da região **estável** ($\rho < 1$) com **oscilação mínima** e **atendimento do SLO**.

MERCADO, CASES E TENDÊNCIAS

O KEDA ativa pods adicionais (linha laranja, escala à direita) para consumir a fila, resultando na queda do backlog (linha azul) e na consequente redução do tempo de resposta médio (linha verde). Após esvaziar a fila, o sistema escala gradualmente de volta para 0 pods, mantendo o tempo de resposta baixo. Este exemplo mostra como o autoscaling reage a bursts de carga mantendo desempenho estável.

O **KEDA** rapidamente deixou de ser apenas uma curiosidade para se tornar uma peça fundamental em arquiteturas nativas de nuvem. Um case real ilustrativo vem da adoção do KEDA no **Azure Kubernetes Service (AKS)** para habilitar processamento de eventos orientado a filas. Imagine uma aplicação de processamento de pedidos de e-commerce: quando uma promoção relâmpago entra no ar, milhares de pedidos são inseridos em uma fila do Azure Service Bus em poucos minutos.

Em vez de superprovisionar continuamente pods de processamento de pedidos, equipes no Azure AKS configuraram um ScaledObject para monitorar o comprimento dessa fila. Assim que os pedidos entram (evento), o KEDA desperta os workers e escala a quantidade de pods proporcionalmente ao ritmo de chegada – garantindo que os pedidos sejam confirmados rapidamente mesmo sob explosão de demanda. Terminado o pico, os pods extras são desligados automaticamente, voltando talvez a zero durante a madrugada.

Esse caso real trouxe **economia de custos** (pagando apenas pelos recursos nos minutos de pico) e simplificação operacional, e inspirou implementações semelhantes em outras nuvens. Em paralelo, na **AWS EKS**, diversas empresas integram o KEDA para autoscaling com Amazon SQS, AmazonKinesis e métricas do CloudWatch. Por exemplo, um cenário fictício, mas verossímil: uma startup mantém um cluster EKS para processar dados de IoT de dispositivos espalhados pelo mundo.

Os dados chegam em rajadas via filas do SQS; usando o KEDA, eles escalam os pods de ingestão conforme o backlog dessas filas ou conforme métricas agregadas no Amazon Managed Prometheus. A própria AWS reconhece essa abordagem, oferecendo guias de boas práticas para integrar o KEDA ao EKS com fontes como SQS e Kafka, combinando-o ao ecossistema (Prometheus/Grafana para visualização

de métricas, OpenTelemetry para traces etc.) para alcançar **autoscaling orientado a eventos na AWS** com sucesso.

É interessante notar que, embora tenha nascido de uma parceria Microsoft/Red Hat, o KEDA é agnóstico de provedor – a **portabilidade** é um atrativo: o mesmo padrão de autoscaling pode ser reproduzido no AKS, EKS, GKE ou clusters on-premises, bastando configurar os devidos scalers.

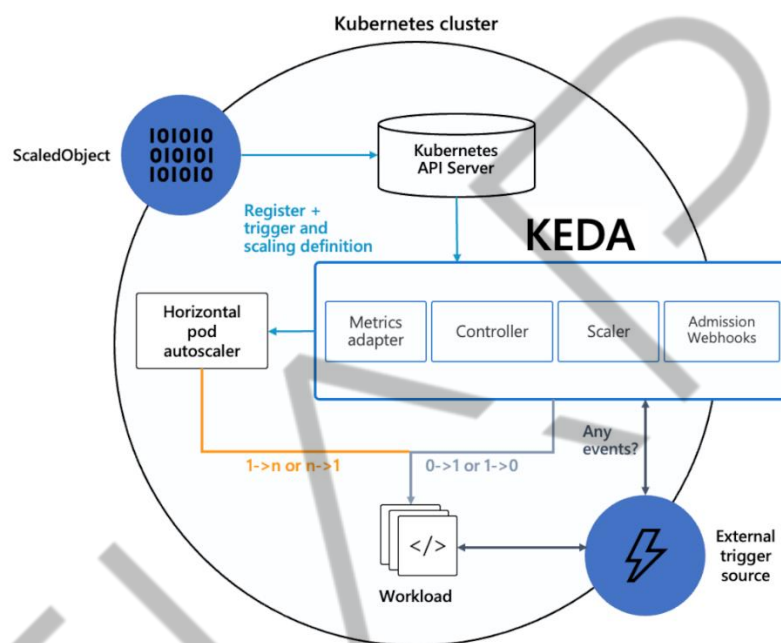


Figura 1 - Diagrama de arquitetura operacional do KEDA
Fonte: Microsoft Learn (s.d.)

O fluxo mostra como eventos externos (ex.: mensagens em fila Amazon SQS ou métricas do Prometheus) são capturados pelo KEDA (via scalers específicos). O KEDA atua como operador no cluster Kubernetes, rodando um Metrics Adapter gRPC e um controlador. Ele observa os triggers definidos nos ScaledObjects, calcula métricas customizadas e as expõe para o HPA.

O Horizontal Pod Autoscaler então decide escalar os pods do Deployment de acordo com essas métricas. O resultado é um loop de realimentação: eventos externos influenciam o HPA via KEDA, que por sua vez ajusta o número de pods no cluster. Em conjunto com monitoramento (Prometheus, Grafana) e componentes de telemetria (AWS OpenTelemetry Collector), obtém-se um sistema completo de autoescalamento com observabilidade.

No horizonte de tendências, vemos o KEDA evoluindo para suportar cenários cada vez mais **orientados a SLOs e custos**. Escalonamento controlado por SLO significa que o alvo não é apenas uma métrica técnica (como QPS ou backlog), mas sim um indicador de qualidade de serviço – por exemplo, manter a **latência abaixo de X ms** ou o **percentual de erros abaixo de Y%**.

Já existem iniciativas combinando o KEDA com controllers de SLO (como o Keptn ou Sloth) para acionar escalonamento somente quando uma métrica de latência viola um objetivo. Isso evita escalonamentos desnecessários e alinha recursos com experiências do usuário. Em paralelo, práticas de **FinOps** estão influenciando o autoscaling: a ideia de *autoscaling serverless* foca em escalar não só para atender demanda, mas para **otimizar custos** dinamicamente – por exemplo, reduzindo agressivamente pods de um workload de baixo priority durante horários de pico de outra aplicação, para economizar em budget global do cluster.

O KEDA, ao facilitar scale-to-zero e escalonamento granular, é peça-chave para implementar essas otimizações de custo cross-workload. Algumas organizações já simulam o custo de cada decisão de escalonamento (via preços de cloud) e utilizam isso como mais um fator nos algoritmos de autoscaling – um conceito emergente de *autoscaling financeiro*.

Outra frente importante é a extensibilidade do KEDA via **scalers personalizados**. A comunidade tem desenvolvido novos scalers para fontes de eventos proprietárias ou não suportadas nativamente, aproveitando o fato de que o KEDA pode carregar **External Scalers** implementados como serviços **gRPC**[\[3\]](#).

Ou seja, qualquer pessoa pode escrever, em qualquer linguagem, um microserviço que implemente a interface gRPC esperada (métodos como `IsActive` e `GetMetric`) e apontar o KEDA para esse serviço. O KEDA irá tratá-lo como mais um scaler, chamando-o periodicamente. Isso abriu espaço para integrações interessantes – por exemplo, escalar com base em consultas ao Azure Cosmos DB, ou com base em webhooks HTTP (há scalers externos que expõem endpoints e quando recebem um trigger externo acionam o KEDA via push, invertendo o modelo de polling).

O uso de gRPC aqui garante performance e baixa latência na comunicação entre o KEDA e esses scalers customizados, mesmo rodando fora do processo do operator. Assim, vemos uma tendência de **expansão do ecossistema** de scalers do

KEDA, cobrindo cada vez mais casos de uso específicos da indústria (desde mensagerias financeiras, fluxos de processamento de vídeo, até sensores de IoT).

Em síntese, o KEDA está se consolidando no mercado como o **padrão de facto para escalonamento event-driven no Kubernetes**, muitas vezes habilitando casos de uso serverless em clusters tradicionais. Empresas já reportam reduções drásticas de custo ao migrar workloads batch para um modelo com KEDA que os deixa zerados quando ociosos, e ganhos de resiliência ao lidarem melhor com bursts imprevisíveis. A convergência de preocupações de SLO e custo aponta para autoscalers cada vez mais inteligentes e multi-variáveis – e o KEDA, com sua flexibilidade de triggers e integração nativa com o HPA, está preparado para ser a base sobre a qual essas camadas de inteligência serão construídas.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos a fundo o tema **escalabilidade de aplicações com KEDA** no Kubernetes. Iniciamos entendendo o **porquê do KEDA** – vimos que o HPA tradicional apresenta limitações frente a workloads orientados a eventos, falhando em escalar a zero e em reagir a métricas externas como tamanho de filas. Descobrimos como o KEDA surgiu para suprir essas lacunas, permitindo **autoscaling baseado em eventos** com suporte a dezenas de fontes (Prometheus, RabbitMQ, Kafka, etc.) e introduzindo o valioso mecanismo de **scale-to-zero**, crucial para economizar recursos em períodos de inatividade.

Em seguida, partimos para um **Hands On** prático, criando um ScaledObject em Rust usando a biblioteca kube-rs. Aprendemos como configurar um Deployment alvo e triggers como o do RabbitMQ, definindo parâmetros como polling interval, cooldown e limites de réplicas para controlar o comportamento de escala. Vimos o ciclo completo de escalonamento: do momento em que uma mensagem chega e ativa pods, passando pelo HPA escalando de 1 para N com base na métrica do backlog, até o rescalonamento para zero após um período sem eventos.

Depois, mergulhamos em conceitos avançados na seção **Saiba Mais**, aplicando *teoria de filas* para modelar o autoscaling. Introduzimos a Lei de Little para relacionar backlog e tempo de resposta, derivamos condições de estabilidade ($\lambda < c\mu$) para evitar sobrecarga e discutimos como atrasos e limiares inadequados podem causar *oscilações* no número de pods. Aprofundamos como técnicas de controle (histerese, janelas de estabilização, tolerância a jitter) são empregadas pelo KEDA/HPA para garantir um escalonamento suave e estável, mesmo sob cargas instáveis.

Por fim, examinamos **cases de mercado e tendências**: entendemos, com exemplos, como organizações estão usando o KEDA em produção no AKS e EKS para habilitar autoscaling serverless e orientado a SLO, e apontamos para o futuro – incluindo a integração de metas de negócio (custos, SLOs) e extensões via gRPC para cobrir novos domínios de eventos. Com isso, você deve ter consolidado não apenas *como* usar o KEDA, mas também *quando* e *por que* ele é tão útil, dominando os fundamentos teóricos que embasam suas decisões de autoscaling.

Aprendemos que escalabilidade no Kubernetes vai muito além de CPU e memória: com o KEDA, podemos escalar aplicações da forma **certa**, no **momento certo**, pelo **motivo certo** (a chegada de um evento), mantendo sistemas responsivos e custos sob controle. **Bons deploys!**

EMEND

REFERÊNCIAS

AMAZON WEB SERVICES. **Guidance for Event-Driven Application Autoscaling with KEDA on Amazon EKS**. AWS Solutions Library. Disponível em: <https://aws.amazon.com>. Acesso em: 7 set. 2025.

BOIE, G. **Kubernetes Workloads Autoscaling with KEDA**. Medium. [s.d.]. Disponível em: <https://medium.com>. Acesso em: 7 set. 2025.

CHANDY, K. M.; NEUSE, D. **Linearizer: A Dynamic Load Balancing Technique**. ACM Transactions on Computer Systems, v. 2, n. 3, p. 231–257, 1982.

CISSE, I. **In-Depth Comparative Analysis: Horizontal Pod Autoscaler (HPA) vs. Kubernetes Event-driven Autoscaler (KEDA)**. Medium. 2025 Disponível em: <https://medium.com>. Acesso em: 7 set. 2025.

KEDA CONTRIBUTORS. **External Scalers**. KEDA Documentation. 2021. Disponível em: <https://keda.sh>. Acesso em: 7 set. 2025.

KEDA DOCUMENTATION. **RabbitMQ Queue Scaler**. KEDA v2.17 Docs. 2023. Disponível em: <https://keda.sh>. Acesso em: 7 set. 2025.

LAKSHMAN, S. **Implementing Event-Driven Autoscaling with KEDA on AKS**. Medium – CareerByteCode. 2025. Disponível em: <https://medium.com>. Acesso em: 7 set. 2025.

LITTLE, J. D. C. **A Proof for the Queuing Formula: $L = \lambda W$** . Operations Research, v. 9, n. 3, p. 383–387, 1961.

LUKŠA, M. **Kubernetes in Action**. Shelter Island: Manning Publications, 2018. ISBN 978-1-61729-372-6.

PALAVRAS-CHAVE

Kubernetes. Autoscaling. KEDA.

EMSE

The background is a dark teal color with a complex network of thin, light teal lines and small circular nodes. These lines and nodes are scattered across the page, creating a sense of a digital or technological environment. Some lines are vertical, while others are horizontal or diagonal. The nodes are small dots, some of which are connected by lines, forming a network-like structure. The overall effect is a futuristic and high-tech aesthetic.

POSTECH